

AgentForge Pre-Search Document

Youssef - Gauntlet G4 Week 2

Date: February 2026

Phase 1: Define Your Constraints

1. Domain Selection

Domain: Finance (Ghostfolio fork)

Use cases: - Portfolio risk analysis - ESG/ethical compliance screening - Tax-loss harvesting - Rebalancing recommendations - Market data integration

Verification requirements: - Market data accuracy (cross-check sources) - Portfolio math correctness (rebalancing preserves value) - ESG compliance against standards database - Tax calculations follow IRS rules

Data sources: Ghostfolio API, Yahoo Finance, ESG standards database, IRS tax code

Why Finance: Understand finance basics, can dogfood with real data, math-based verification (objective).

2. Scale & Performance

Query volume: 50-100 queries/day

Acceptable latency: - Single-tool: <5 seconds - Multi-step: <15 seconds

Concurrent users: 5-10 (showcase)

Cost constraints: \$0-10 for LLM calls

Performance targets: >95% tool success, >80% eval pass, <5% hallucination rate.

3. Reliability Requirements

Cost of wrong answer: - Wrong stock price: Bad investment decision - Wrong risk calc: Portfolio mismanagement - Wrong ESG status: Trust erosion - Wrong tax advice: IRS penalties

Non-negotiable verification: 1. Market data accuracy (cross-reference 2+ sources) 2. Calculation correctness (rebalancing preserves value) 3. ESG compliance (authoritative source) 4. Tax rules (30-day wash sale) 5. Source attribution (every claim cited)

Human-in-the-loop: Recommendations >\$10K, conflicting sources, tax >\$1K

Compliance: Document "informational only, not financial advice", audit trail.

4. Team & Skill Constraints

Team: Solo

Agent framework: None (learning LangGraph)

Finance experience: Moderate (understand risk, diversification, allocation)

Development: AI-assisted (Cursor + Claude Code)

Biggest risk: Ghostfolio complexity (mitigate: use API, not fork internals)

Learning curve: LangGraph (4-6h), Ghostfolio API (2-3h), formulas (3-4h)

Phase 2: Architecture Discovery

5. Agent Framework Selection

Framework: LangGraph

Architecture: Single agent with state transitions

State management: Portfolio → risk metrics → compliance checks → recommendations

Tool integration: Moderate (Ghostfolio API, Yahoo Finance, ESG database)

Why LangGraph: Portfolio analysis is a state machine (fetch → analyze → check → optimize).
Conditional logic built-in.

6. LLM Selection

LLM: Claude Sonnet 3.5

Function calling: Required

Context window: 200K tokens (can hold full portfolio)

Cost per query: ~\$0.003-0.01

Why Claude: Structured outputs, good at following instructions (important for financial accuracy), familiar from Week 1.

Dev cost estimate: ~\$10 total (500 test queries + 50 eval runs).

7. Tool Design

Tools needed (6 total):

1. **portfolio_risk_analysis** - Calculate volatility, Sharpe ratio, beta, concentration risk
2. **compliance_checker** - ESG/ethical screening, flag violations, suggest alternatives
3. **rebalancing_suggestions** - Target allocation, specific buy/sell actions
4. **tax_loss_harvesting** - Identify opportunities, verify 30-day wash sale rule
5. **market_data_fetch** - Real-time prices, PE ratios, dividends (Yahoo Finance)
6. **portfolio_giving_calculator** - Charitable donation planning, tax deduction estimates

External APIs: Yahoo Finance (primary), Alpha Vantage (backup), ESG standards database

Mock vs real: Market data real, Ghostfolio real, ESG standards CSV for MVP

Error handling: Graceful failures, cached data fallbacks, stale data warnings

8. Observability Strategy

Tool: LangSmith

Metrics: Trace logging, latency breakdown, token usage, tool success rate, hallucination detection

Real-time monitoring: Yes (dashboard during demo)

Cost tracking: Project to 100/1K/10K users

Why LangSmith: Native LangGraph integration, free tier (5K traces/month), visual debugging.

9. Eval Approach

Correctness measurement: - Calculations: Programmatic verification (formulas) - Market data: Cross-reference authoritative sources - ESG compliance: Compare to ground truth database - LLM-

as-judge: For subjective recommendation quality

Eval dataset: 50 test cases - 15 risk analysis - 10 ESG compliance - 10 rebalancing - 10 tax harvesting - 5 adversarial

Automated vs human: 80% automated (LLM-as-judge), 20% manual review

Pass rate target: 82-85% overall

10. Verification Design

Claims to verify: 1. Market data (cross-check Yahoo + Alpha Vantage, flag >5% discrepancy) 2. Calculations (rebalancing must preserve total value) 3. ESG status (compare agent vs database) 4. Source attribution (every claim cites source + timestamp) 5. Confidence scoring (high >90%, medium 70-90%, low <70%)

Escalation triggers: >\$10K recommendations, conflicting sources, stale data (>24h)

| Phase 3: Post-Stack Refinement

11. Failure Mode Analysis

Tool failures: - Yahoo Finance down → Use Alpha Vantage - All APIs down → Use cached (mark stale) - Ghostfolio timeout → Retry with backoff

Ambiguous queries: - “Analyze portfolio” → Clarify: risk? compliance? both? - “Is X ethical?” → Explain complexity if gray area

Rate limiting: Yahoo 2K req/hour, Claude 50 req/min

Graceful degradation: Never crash, always provide fallback (cached data + warning).

12. Security Considerations

Prompt injection: Sanitize input (500 char limit, block “ignore previous” patterns)

Data leakage: Session-scoped state, don't log full portfolios

API keys: Claude API server-side only, Ghostfolio backend only

SEC compliance: Disclaimer “informational only, not financial advice”, no buy/sell recommendations

13. Testing Strategy

Unit tests: Calculate Sharpe ratio, rebalancing math, ESG lookup, wash sale logic (4h)

Integration tests: Full workflow (fetch → analyze → check → suggest) (3h)

Adversarial: Hallucinated prices, invalid tickers, prompt injection (2h)

Regression: Not for showcase (document as production requirement)

14. Open Source Planning

Release: ESG/ethical investment screening toolkit for LangGraph

Licensing: MIT

Documentation: README, tutorial, API reference, examples

Community: r/LangChain, LinkedIn, Twitter with demo

Why: Novel contribution (ESG screening for portfolio agents), serves socially-conscious investors.

15. Deployment & Operations

Hosting: - Frontend: Vercel - Backend: Railway (LangGraph agent) - Database: PostgreSQL on Railway

CI/CD: Git push → auto-deploy (both)

Monitoring: LangSmith (traces), Railway logs, Vercel analytics

Rollback: Vercel instant, Railway manual

16. Iteration Planning

Feedback: Thumbs up/down, “Report incorrect data” button

Eval-driven cycle: Run eval → identify failures → fix → re-run → measure improvement

Feature priority: P0 (risk + compliance), P1 (ESG screening), P2 (tax + rebalancing)

Long-term: Not applicable for showcase (document approach only)

Complete Tech Stack

Layer	Technology	Time	Why
Framework	LangGraph	6h	State machines for portfolio workflows
LLM	Claude Sonnet 3.5	0h	Structured outputs, cost-effective
Observability	LangSmith	2h	Native integration
Tools	6 finance tools	24h	Risk, compliance, tax, rebalancing
Verification	5 layers	10h	Data, calculations, compliance
Eval	50 cases	6h	82%+ pass target
Deployment	Vercel + Railway	2h	Simple, familiar

Total: ~50 hours setup

Remaining: ~30 hours features/polish

Document prepared: February 2026

Pre-search time: 2 hours